Chapter 2 | Boolean Logic & Digital Electronics

2.1 Boolean Algebra — Laws, Duality and De Morgan

Boolean algebra: the **algebra of logic**, given by George Boole. Every variable and every expression takes only a **Boolean value** — 0 or 1. It is the algebra used to **express logic gates**, to write the input-output relationship of a logic diagram in algebraic form, and to **design digital circuits** (both tested statements true). In Boolean algebra, **+ means OR** (logical addition) and **· means AND** (logical multiplication — the same symbol as arithmetic multiplication).

Law	OR form	AND form
Identity	A + 0 = A	A · 1 = A
Annulment	**A + 1 = 1**	A · 0 = 0
Idempotent	**A + A = A**	**A · A = A** (no squaring in Boolean algebra)
Complement	**A + A′ = 1** (never A — the invalid-rule trap)	A · A′ = 0
Involution	(A′)′ = A — an even number of complements cancels: ((((x′)′)′)′ = x	—
Absorption	**A + AB = A**	**A(A + B) = A**; also yz + z = z; and x′ + xy′ = x′ + y′
Distributive	**A + BC = (A + B)(A + C)** — OR distributes over AND (arithmetic cannot do this)	A(B + C) = AB + AC

Duality: to form the **dual** of an expression, **interchange all 0s and 1s, and all + and · signs** — the variables are NOT complemented (complementing them too gives the complement, not the dual).

De Morgan's theorems: the bridge between AND and OR through complementation — used for **simplifying Boolean expressions and for NAND-only implementation** (both). First theorem: the complement of an **OR** equals the **AND** of the complements; second: complement of an AND equals the OR of the complements:

$$(A + B)' = A' \cdot B' \qquad (A \cdot B)' = A' + B'$$

Two counting facts: the **degree** of a term is its number of literals (A′B′C′D′ has degree **4**), and with two variables there are 2^{2^2} — that is 2^4 = **16 possible Boolean functions** (each of the 4 truth-table rows independently 0 or 1).

Worked Simplifications from the Papers

(1) x + xy = **x**; x(x + y) = **x**; y·z + z = **z**; AB(A + B) = **AB** (idempotent + absorption).

(2) (1 + A)(A + B)($\bar{A}$ + B): the first factor is 1; (A + B)(A′ + B) = **B**.

(3) E(E + F) + DE + D(E + F) = E + EF + DE + DF = **E + DF** (E absorbs its multiples).

(4) (X + Y + XY)(X + Z): absorb inside first → (X + Y)(X + Z) = **X + YZ** (distributive law).

(5) AB + AB′ + A′C + AC = A + C — **independent of B** (X·Y + X·Y′ = X, twice).

(6) A′C + A′B + AB′C + BC = **C + A′B**. (AB)′(A′ + B)(B′ + B) = (A′+B′)(A′+B)·1 = **A′**. ~(A·B)(~A + B)(~B + B) = **~A** — the same pattern with different symbols.

(7) Complement of F = AB + C′D + B′D: each product becomes a sum — F′ = **(A′+B′)(C+D′)(B+D′)**. Complement of f = (a + bc)(pq + r): f′ = **a′(b′ + c′) + (p′ + q′)r′** (De Morgan layer by layer).

(8) xy + x′z = **(x′ + y)(x + z)(y + z)** by the distributive law — the extra (y + z) is the consensus term.

(9) **xy′ + x′y = XOR; x′y′ + xy = XNOR** — neither reduces to a single variable (so "x′y′ + xy = x" is the invalid identity). XOR(x′, y′) = XOR(x, y) — complementing both inputs changes nothing; complementing ONE input gives XNOR. And xy′ + x′y + x′y′ = **x′ + y′**.

Canonical Forms and the Karnaugh Map

Canonical forms: Sum of Minterms (SOM) and Product of Maxterms (POM). A minterm is a product containing every variable; a maxterm is a sum containing every variable. **SOP consists of minterms; POS consists of maxterms**. Y = AB + BC + AC is an **SOP** expression (OR of ANDs).

Karnaugh map: the graphical simplification tool. Its rows and columns are ordered in **Gray code (00, 01, 11, 10) — NOT plain binary sequence** (the tested false statement); adjacent squares are grouped in **powers of two**, covering as many cells as possible; it simplifies **both SOP and POS** forms. A **don't-care** condition marks an input whose output **does not affect the result** — treat it as 0 or 1, whichever gives bigger groups.

Worked: F(x, y, z) = Σ(0, 2, 4, 5, 6): all four z = 0 cells group to z′; cells 4, 5 group to xy′ → **F = z′ + xy′**.

Exam Points — 2.1

- A + 1 = 1; A·A = A; A + A′ = 1 ('A + A′ = A' is the invalid rule); A + AB = A (absorption); A + BC = (A+B)(A+C); dual = swap operators and constants only.
- De Morgan: (A+B)′ = A′B′ (OR → AND of complements); (AB)′ = A′+B′. Used for simplification AND NAND implementation.
- Degree of A′B′C′D′ = 4; two variables → 16 functions. All nine worked simplifications above are direct paper questions.
- Canonical forms = SOM + POM; POS = maxterms, SOP = minterms. K-map order = Gray code; don't-care = value does not affect output; Σ(0,2,4,5,6) → z′ + xy′.

2.2 Logic Gates

A logic gate takes one or more inputs and produces **exactly one output**. Its input-output relationship is written in tabular form as a **truth table**, which has one row per input combination — **n inputs give 2^n rows** (a 4-input gate of any type → 16 rows). The **NOT gate (inverter)** has one job only: it **inverts the input signal** — and it is not universal by itself.

Gate	Expression	Word rule
AND	y = a · b	1 only when **all** inputs are 1; 0 if **at least one** input is 0. "All true" = AND
OR	y = a + b	1 if **at least one** input is 1; **0 only when all inputs are 0**. Railway crossing ('close if a train on at least one track') = OR
NOT	y = a′	Complements the input
NAND	**(xy)′**	**0 only when all inputs are 1**; so **any low input ⇒ high output**; all-0s → 1

NOR	**(A + B)′ = A′ · B′** (by De Morgan — note: A′ + B′ would be NAND; the paper's own key erred here and the explanation corrects it)	1 only when all inputs are 0
XOR (⊕)	xy′ + x′y	**1 when the inputs are DIFFERENT, 0 when same**; needs a minimum of **2 inputs**
XNOR (⊙)	xy + x′y′	**1 when the inputs are the SAME** — the equality detector; Y = A ⊙ B

XOR — the Most-Asked Gate

(1) Identities: **A ⊕ A = 0** (both XOR inputs tied to B → output 0); A ⊕ 0 = A; **A ⊕ 1 = A′** (XOR with 1 inverts).

(2) The two-way **staircase switch** (light on only when the switches differ) is XOR. Including an **XOR gate turns a binary adder into an adder-subtractor** (it conditionally complements one operand).

(3) Register micro-operation R1 ← R1 ⊕ R2 with R1 = 1010, R2 = 1100 → R1 = **0110**, R2 unchanged. The **mask** micro-operation (clearing selected bits) is the **AND** gate's job — AND with 0 clears, OR with 1 sets.

(4) Three variables: **(A ⊕ B ⊕ C)′ = A ⊕ B ⊙ C** — complementing an XOR chain flips exactly one ⊕ to ⊙.

Symbols and Gate Counting

(1) XOR symbol = OR shape with an **extra curved line** across the inputs. A NOR gate has **two valid drawings**: OR-with-bubble, and AND-with-inverted-inputs (De Morgan as a picture). A **bubbled OR gate = NAND**; a bubbled AND = NOR.

(2) SOP gate count: F = AB + BC + AC needs **one AND per product term** (3) feeding one OR. For an n-input AND: output 1 only for all-1s, 0 if any input is 0.

Universal Gates

Only NAND and NOR are universal — each alone can build NOT, AND, OR and hence any digital circuit (each contains an inversion; AND and OR cannot complement). So "all basic gates cannot be implemented using universal gates" is the false statement, and "digital circuits can be made by repeated use of…" takes **more than one** answer (NAND or NOR).

(1) **A NAND (or NOR) with its inputs tied together is a NOT gate**: (A·A)′ = A′ — the first step of every NAND-only build.

(2) Gate counts: AND from NOR = **3** gates (invert each input, then NOR); AND from NAND = 2; **XOR from NAND = 4 gates; XNOR = 5.**

(3) **SOP → NAND-NAND; POS (product of sums) → NOR-NOR** realisation.

Note: Two stray questions in this band belong to Chapter 10: MAN = collection of LANs over a city + carries data and voice (asked twice; once keyed 'only I', once 'both' — the full networking treatment is in the Networks chapter).

Exam Points — 2.2

✦ One gate, one output. Truth table = tabular I/O form; n inputs → 2^n rows. NOT inverts only. OR: 0 only for all-0s. AND operator = multiplication symbol.

DSSSB – TGT COMPUTER SCIENCE (PRACTICE KING) BOOK

Call/Message: 7011121290 Available @ Amazon: https://amzn.in/d/07E04o9B

✦ NAND = (xy)′, 0 only for all-1s, any low input gives high output. NOR = (A+B)′ = A′B′. XNOR(1,0) = 0; XNOR = same-inputs-high gate.

✦ A⊕A = 0; XOR → staircase switch, adder-subtractor conversion; masking = AND; (A⊕B⊕C)′ = A⊕B⊙C.

✦ Universal = NAND and NOR only. Tied-input NAND = NOT. NOR→AND = 3 gates; XOR needs 4 NANDs. SOP→NAND-NAND, POS→NOR-NOR. Bubbled OR = NAND.

2.3 Combinational Circuits — Adders, MUX, Decoders

A **combinational circuit's output depends only on the present inputs** — no memory. Examples: **encoder, decoder, multiplexer, adder**. Counters, shift registers and clocks contain memory, so they are sequential. The part of the computer that performs addition, subtraction, multiplication, division and **comparison** is the **ALU**.

Adders and Subtractors

The half adder adds two bits; the full adder adds three (two operand bits + carry-in):

$$Half\ adder:\ S = A \oplus B,\quad C = A \cdot B$$
$$Half\ subtractor:\ D = A \oplus B,\quad Borrow = A' \cdot B$$
$$Full\ adder:\ S = x \oplus y \oplus z,\quad C = xy + (x \oplus y)z$$

(1) An **n-bit binary adder needs n full adders**. A full adder = 2 half adders + 1 OR gate; so a **4-bit parallel adder WITHOUT input carry = 7 HA + 3 OR** (the first stage needs only one HA and no OR).

(2) In 2-input gates a full adder = **2 XOR + 2 AND + 1 OR** (two XORs make the sum; the ANDs and OR make the carry).

(3) **Complementary subtraction** = subtraction by the additive approach (add the complement) — why hardware subtracts via 2's complement. The micro-operation R3 ← R1 + R2′ + 1 means **R1 minus R2** (add the 2's complement).

(4) **BCD adder**: each 4-bit input is valid only for 0–9, so of 16 × 16 = 256 input combinations only 10 × 10 = 100 are valid → **156 invalid**.

Multiplexers, Demultiplexers, Decoders, Encoders

$$MUX:\ 2^{\,m} = n\ inputs \qquad DEMUX:\ n = 2^{\,m}\ outputs \qquad (m = select\ lines)$$

(1) **MUX = many-to-one** (selects one of 2^m inputs); **DEMUX = one-to-many** — accepts one input and distributes it over several outputs. MUX advantages: **simplifies logic design, minimises IC package count, and removes the need for logic simplification** — all three.

(2) **Decoder** = one input code → many output lines. The **seven-segment decoder** drives station-clock digit displays. An n-to-2^n decoder produces every minterm, so **any function needs just one OR gate per output**: a **full subtractor = 3-to-8 decoder + 2 OR gates** (one OR for Difference, one for Borrow).

(3) **Encoder** = the reverse (2^n inputs → n-bit code). A **decimal-to-BCD encoder has 10 inputs and 4 outputs**. A **binary-to-Gray code converter is an encoder** (built from XORs).

(4) A **PLA implements logic with an AND array feeding an OR array** (SOP in hardware). PLA: both arrays programmable; PROM: fixed AND; PAL: programmable AND, fixed OR.

Exam Points — 2.3

✦ Encoder/decoder/MUX/adder = combinational; counter/shift register = sequential. ALU does arithmetic + comparison.

✦ HA: S = A⊕B, C = AB. Half subtractor borrow = A′B. FA: S = x⊕y⊕z, C = xy + (x⊕y)z. n-bit adder = n FAs; 4-bit without carry-in = 7 HA + 3 OR; 2-input-gate FA = 2 XOR + 2 AND + 1 OR.

✦ Complementary subtraction = additive method. BCD adder invalid inputs = 156.

✦ MUX 2^m = n; DEMUX n = 2^m; MUX advantages all three. 7-segment decoder for displays; full subtractor = 3-to-8 decoder + 2 OR. Decimal-to-BCD encoder = 10 : 4; binary→Gray converter = encoder. PLA = AND + OR arrays.

2.4 Sequential Circuits — Latches and Flip-Flops

A **sequential circuit requires a clock signal AND memory elements** (both tested statements true) — its output depends on the present inputs and the stored state. A **latch is level-triggered** storage (it also captures multiple bits of a row in parallel for simultaneous reading); a **flip-flop is edge-triggered** and **stores exactly 1 bit**. A **negative level triggered flip-flop changes state while the clock is at the low (negative) level**.

Flip-flop	Behaviour and asked facts
SR (NOR latch)	Set/Reset. **Forbidden input: S = R = 1** — from (0,1) the outputs settle at (1,0); raising inputs to (1,1) drives both outputs to **(0,0)**. (NAND latch's forbidden input is 0,0)
D	Copies D to Q on the clock: **D = 1 → Q = 1; D = 0 → Q = 0**. One data input only, so it **never suffers the race-around condition**
JK	Removes SR's forbidden state; but with **J = K = 1 and the clock held high, the output races (toggles repeatedly)** — cured by edge-triggering or a master-slave arrangement. A JK flip-flop is a building block of **sequential** (not combinational) logic
T (toggle)	**Obtained from JK by tying J = K**. T = 1 → toggles each clock; T = 0 → holds (both statements true)

Multivibrators: bistable = the flip-flop (two stable states); **monostable = one stable + one quasi-stable state** (the one-shot); **astable = free-running — no stable state, two quasi-stable states**. The pairing "astable–flip-flop, bistable–free-running" is both wrong.

(1) **MAR** holds the **address** of the memory location being read or written; the memory buffer register (MBR/MDR) holds the data itself.

(2) **Micro-operations are classified four ways**: register-transfer, arithmetic, logic and shift. Worked arithmetic micro-op: A = 01110110, B = 10000101; A ← A + B gives A = **11111011**, and B ← B + 1 gives B = **10000110**.

(3) **Parallel synchronous load of a byte needs exactly ONE clock pulse** (serial load would need eight — one per bit).

(4) **Propagation delay** determines the time devices need to coordinate use of the bus.

Exam Points — 2.4

✦ Sequential = clock + memory. Flip-flop stores 1 bit; latch is level-triggered and reads a row in parallel; negative-level-triggered = active while clock low.

✦ NOR latch forbidden (1,1) → outputs (0,0). D: output follows D (1; 0); clocked D has no race condition. J = K tied → T flip-flop; T = 1 toggles, T = 0 holds.

✦ MAR = address register. Micro-operations: 4 classes; the A + B / B + 1 worked values. Parallel load = 1 clock pulse. Propagation delay coordinates the bus.

2.5 Registers and Counters

Shift register: a chain of flip-flops named by input-mode + output-mode: SISO, SIPO, PISO, **PIPO** — four D flip-flops loaded together and read together = **Parallel-In Parallel-Out**.

Counter: the sequential device that **stores and displays the number of times an event has occurred**, usually against a clock. **n flip-flops count from 0 to $2^n - 1$.**

Ripple (asynchronous) counter	Synchronous counter
Only the FIRST flip-flop takes the external clock; each output clocks the next — the count 'ripples'	All flip-flops share one common clock
Delay accumulates: total = **n × t_pd**; it **skips a count when that total exceeds the clock period** — 10-bit counter at 10 MHz (period 100 ns) skips if each FF delay reaches **10 ns**	Delay = **one flip-flop's delay** — 2-bit counter with 25 ns flip-flops changes state in just **25 ns**

(1) **Cascading multiplies the moduli**: Mod-6 × Mod-8 = **Mod-48**; MOD-3 × 4 × 5 = 60, so a 12 MHz clock leaves the chain at 12 MHz ÷ 60 = **200 kHz**.

(2) **Ring counter** feeds Q of the last stage back to the first; the **twisted-ring (Johnson) counter feeds the COMPLEMENT output of the last stage to the D input of the first**.

(3) In a ripple counter, **if the uncomplemented outputs count up, the complemented outputs count down** (true) — read either set.

Exam Points — 2.5

✦ PIPO = parallel in, parallel out. Counter counts events; n FFs → 0 to 2^n-1.

✦ Ripple = asynchronous, first FF clocked externally, delays add (10 ns skip threshold worked); synchronous delay = one FF (25 ns worked).

✦ Cascade → multiply moduli (Mod-48; 200 kHz worked). Johnson counter = complemented feedback. Complemented outputs count down.

2.6 Logic Families and Integrated Circuits

Transistor: a **controlled electronic switch fabricated from a semiconductor** — the basic element of every gate. **Digital circuits process discrete (0/1) levels**, not continuous voltages — that is exactly what makes them noise-robust; the true statement in the exam item is that they are "not suitable" for directly representing continuous quantities.

Integration scale by gate count per chip:

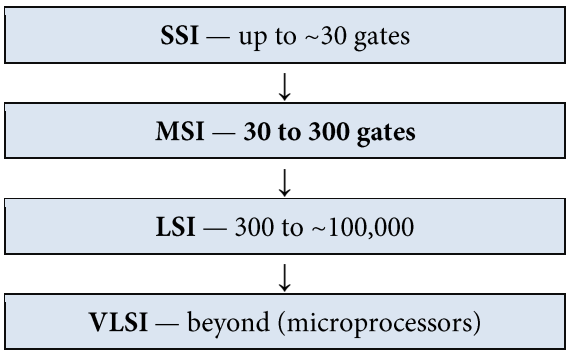


Fig. 2.1 — IC integration levels. The 30–300 band is MSI — the asked one.

Family	Asked facts
RTL / DTL / DCTL	Resistor– / Diode– / **Direct-Coupled Transistor Logic** (names decode literally); **RTL is a SATURATED bipolar family**
TTL	Transistor-Transistor Logic — the classic 74-series
ECL	Emitter-Coupled Logic — **non-saturated**, hence **the FASTEST logic family** (and the hungriest)
MOS / CMOS	**Basic gates of the MOS family: NAND and NOR** (inversion is free in MOS); CMOS = lowest power

Remember: **Figure of merit of an IC = propagation delay × power** — the speed-power product; smaller is better.

Exam Points — 2.6

✦ Transistor = semiconductor-controlled switch. 30–300 gates = MSI. RTL saturated, ECL non-saturated and fastest; DCTL = Direct-Coupled Transistor Logic; MOS basic gates = NAND + NOR.

✦ Figure of merit = delay × power.

2.7 Complements in Any Base and Binary Arithmetic

The 2's-complement idea generalises to every base r. For an n-digit number N:

$$(r-1)'s\ complement = subtract\ each\ digit\ from\ (r-1) \qquad r's\ complement = r^n - N$$

(1) 15's complement (hex) of C0D0: subtract each digit from F → **3FF2F**.

(2) 10's complement of 20100: $10^5 - 20100$ = **79900**.

(3) 6's complement (base 7) of $(1543)_7$: subtract each digit from 6 → **5123**.

(4) Dividing a binary number by 2 = shift one place right: 10100 ÷ 2 = **01010**.

(5) Mixed-base arithmetic goes through decimal: $(10110)_2 \times (10011)_2 = 22 \times 19 = 418 =$ **$(110100010)_2$**; $(3564)_8 + (323232)_4 = 1908 + 3822 = 5730 =$ **$(1662)_{16}$**. Base-solving: $(1101)_x = (241)_{16} = 577 \rightarrow x^3 + x^2 + 1 = 577 \rightarrow$ **x = 8**.

(6) BCD of $(1000)_{10}$ needs 16 bits (4 digits × 4 bits). **Guard bits avoid unnecessary loss of the LSB** (precision) during floating-point arithmetic — guard, round and sticky bits. **EBCDIC = Extended Binary Coded Decimal Interchange Code** (recap from Chapter 1).

Exam Points — 2.7

✦ (r−1)'s complement = digit-wise subtraction from r−1; r's complement = $r^n - N$. Worked: 3FF2F, 79900, 5123.

✦ ÷2 = right shift (10100 → 01010). Mixed bases: convert to decimal first (worked products and sums above). BCD of 1000 = 16 bits; guard bits protect the LSB.

Quick Revision — 30 points

DSSSB – TGT CS (PRACTICE KING) – SUPPORT MATERIAL [TOPPER'S CHOICE]

1. Boolean algebra = algebra of logic (Boole); + is OR, · is AND; expressions take only Boolean values; used to design digital circuits.
2. Laws: A+1 = 1; A·0 = 0; A+A = A; A·A = A; A+A′ = 1; A·A′ = 0; (A′)′ = A; A+AB = A; A(A+B) = A; A+BC = (A+B)(A+C).
3. Dual = swap + and ·, swap 0 and 1, variables untouched. De Morgan: (A+B)′ = A′B′; (AB)′ = A′+B′.
4. Degree = number of literals; 2 variables → 16 functions. xy′+x′y = XOR; xy+x′y′ = XNOR; complementing one XOR input gives XNOR, both inputs gives XOR again.
5. Key simplifications: x+xy = x; x(x+y) = x; yz+z = z; AB(A+B) = AB; (X+Y+XY)(X+Z) = X+YZ; AB+AB′+A′C+AC free of B; F′ of AB+C′D+B′D = (A′+B′)(C+D′)(B+D′).
6. Canonical forms: sum of minterms, product of maxterms; SOP↔minterms, POS↔maxterms. K-map order = Gray code; groups = powers of 2; don't-care = output irrelevant; Σ(0,2,4,5,6) = z′+xy′.
7. Gate = one output; truth table has 2^n rows. NOT only inverts. OR: 0 only for all zeros; AND: 1 only for all ones; 'at least one' = OR.
8. NAND = (xy)′, 0 only for all-1s, any low input → high output. NOR = (A+B)′ = A′B′. XOR = difference (min 2 inputs); XNOR = equality (A⊙B).
9. A⊕A = 0; A⊕1 = A′; staircase switch = XOR; adder+XOR = adder-subtractor; masking = AND; R1←R1⊕R2 worked (0110); (A⊕B⊕C)′ = A⊕B⊙C.
10. Universal gates = NAND and NOR only; tied-input NAND = NOT; AND from NOR = 3 gates; XOR = 4 NANDs, XNOR = 5; SOP→NAND-NAND, POS→NOR-NOR; bubbled OR = NAND.
11. Combinational (no memory): encoder, decoder, MUX, adder. Sequential: counter, register, clock. ALU computes and compares.
12. Half adder: S = A⊕B, C = AB. Half subtractor: D = A⊕B, Borrow = A′B. Full adder: S = x⊕y⊕z, C = xy+(x⊕y)z.
13. n-bit adder = n full adders; 4-bit without input carry = 7 HA + 3 OR; FA in 2-input gates = 2 XOR + 2 AND + 1 OR; complementary subtraction = add the complement; BCD adder invalid inputs = 156.
14. MUX: 2^m inputs, m selects, many→one; DEMUX: one→many, n = 2^m; MUX advantages: simpler design, fewer ICs, no manual simplification.
15. Decoder: code→lines; 7-segment decoder for digit displays; full subtractor = 3-to-8 decoder + 2 ORs. Encoder: decimal→BCD = 10:4; binary→Gray converter = encoder. PLA = AND array + OR array.
16. Sequential circuits need clock + memory. Latch = level-triggered (parallel row capture); flip-flop = edge-triggered, stores 1 bit; negative level = active while clock low.
17. NOR latch forbidden (1,1) → (0,0); NAND latch forbidden (0,0). D copies D to Q; no race. JK race-around at J = K = 1 with clock high; master-slave cures it. J = K → T; T = 1 toggle, T = 0 hold.
18. Multivibrators: bistable = flip-flop; monostable = 1 stable + 1 quasi; astable = free-running, 2 quasi-stable.
19. MAR = address, MBR = data. Micro-operations: 4 classes (register-transfer, arithmetic, logic, shift). R3←R1+R2′+1 = R1−R2. Parallel load = 1 pulse.
20. Registers: SISO/SIPO/PISO/PIPO by I/O mode. Counter counts events; n FFs → 0 to 2^n-1.
21. Ripple counter = asynchronous, first FF clocked, delays add (skip when n×t_pd > period; 10 ns worked). Synchronous = common clock, delay of one FF (25 ns worked).
22. Cascaded moduli multiply: 6×8 = 48; 12 MHz ÷ (3×4×5) = 200 kHz. Ring feeds Q back; Johnson (twisted ring) feeds Q′ back. Complemented outputs count down.

DSSSB – TGT COMPUTER SCIENCE (PRACTICE KING) BOOK

Call/Message: 7011121290 Available @ Amazon: https://amzn.in/d/07E04o9B

23. Transistor = semiconductor switch. Digital = discrete levels. SSI < MSI (30–300) < LSI < VLSI.

24. RTL saturated; ECL non-saturated = fastest; CMOS lowest power; MOS basic gates NAND + NOR; DCTL = Direct-Coupled Transistor Logic. Figure of merit = delay × power.

25. (r–1)'s complement = subtract digits from r–1: 15's of C00D0 = 3FF2F; 6's of $(1543)_7$ = 5123. r's complement = r^n–N: 10's of 20100 = 79900.

26. Binary ÷2 = right shift. Mixed-base sums/products via decimal: 22×19 = 418; $(3564)_8+(323232)_4 = (1662)_{16}$; $(1101)_x = (241)_{16} \rightarrow x = 8$.

27. BCD of 1000 = 16 bits. Guard bits protect the LSB. EBCDIC = Extended Binary Coded Decimal Interchange Code.

28. Half of the gate questions are truth-table identifications: 1 only for all-1s = AND; 1 for any 1 = OR; 0 only for all-1s = NAND; XOR gives 1, 0 for different, same inputs.

29. K-map false-statement trap: rows/columns are in Gray code, never plain binary sequence.

30. NOR-gate question with key A′+B′: the book's own explanation corrects it to A′B′ — De Morgan decides.

DSSSB – TGT COMPUTER SCIENCE (PRACTICE KING) BOOK

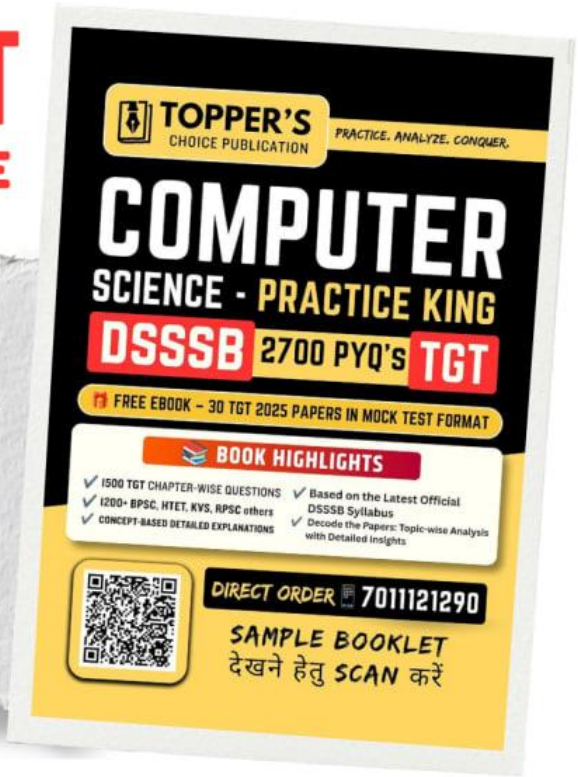
Call/Message: 7011121290 **Available @ Amazon:** https://amzn.in/d/07E04o9B

7011121290

STUDY THE PATTERN, NOT THE SYLLABUS

DSSSB -TGT
COMPUTER SCIENCE

रटने वाली नहीं,
EXAM को DECODE करने वाली
BOOK!
हर PYQ के साथ
Concept + Trend Analysis



Scan करें और Amazon से
तुरंत ऑर्डर करें

DSSSB – TGT COMPUTER SCIENCE (PRACTICE KING) BOOK

Call/Message: 7011121290 Available @ Amazon: https://amzn.in/d/07E04o9B